

Quantitative Analysis of NVIDIA Stock Directional Forecasting

DATA 210 — Statistical Methods I

Ethan Wong

University of California, Irvine

Donald Bren School of Information and Computer Sciences

March 16, 2026



Abstract

This project investigates whether historical price and volume data can be used to predict the daily directional movement of NVIDIA Corporation (NVDA) stock over a decade-long period from January 2016 to December 2025. As one of the most closely watched equities in the market, NVIDIA presents a particularly compelling case study for testing the limits of quantitative forecasting. The analysis is grounded in two competing theoretical frameworks: the Efficient Market Hypothesis (EMH), which holds that all publicly available information is already priced in, and the Adaptive Markets Hypothesis (AMH), which allows for temporary inefficiencies in highly volatile assets. Understanding which framework better describes NVIDIA’s behavior has practical implications for retail investors, quantitative analysts, and anyone evaluating the feasibility of algorithmic trading strategies on high-profile large-cap equities.

The dataset consists of 2,514 daily OHLCV observations sourced from Kaggle, covering a period that spans NVIDIA’s transformation from a gaming hardware company to the dominant supplier of AI accelerators. After stationarity testing via the Augmented Dickey-Fuller (ADF) test and log-return transformation, technical indicators were engineered as classification features, including RSI, MACD, Bollinger Bands, and rolling volatility measures. Four models were trained and evaluated on next-day directional prediction: ARIMA, Logistic Regression with L2 regularization, Random Forest, and a Soft Voting Ensemble combining the Logistic Regression and Random Forest models. All four models converged on accuracy figures between 51% and 55%, barely above the 50% random-chance baseline. Random Forest produced the most balanced result with the highest precision (56.9%), while Logistic Regression and ARIMA both collapsed toward predicting “up” on nearly every day, exploiting the asset’s bull-market base rate rather than extracting genuine signal. The ensemble failed to correct for this bias, instead inheriting Logistic Regression’s upward skew. Taken together, the results are broadly consistent with the EMH: for a heavily covered, highly liquid stock like NVIDIA, price history alone provides insufficient signal for systematic directional forecasting.

Introduction

The academic foundation of stock forecasting begins with the Efficient Market Hypothesis (EMH), famously championed by Eugene Fama in the 1960s. The EMH states stock prices follow a “random walk,” meaning all known information is already “priced in,” and future movements are impossible to predict consistently. Under this framework, any attempt to forecast NVIDIA’s daily direction would be no more effective than a coin flip. However, as the field of behavioral finance emerged, researchers began to identify “market anomalies”, persistent patterns driven by human psychology, such as overreaction to news or herding behavior. This led to the Adaptive Markets Hypothesis (AMH), which suggests that while markets are generally efficient, they undergo periods of temporary inefficiency. For a high-volatility stock like NVDA, these periods of inefficiency offer a window for quantitative models to capture “alpha” by identifying recurring structural patterns in historical data.

Literature Review

In statistical finance, the primary challenge for modeling stock price movements is that financial time series are rarely stationary, meaning they are not independent and identically distributed (i.i.d.). As noted by Cont (2001), financial returns exhibit characteristic “stylized facts”: fat tails, volatility clustering, and little to no autocorrelation in returns. These properties motivate the need for stationarity testing, typically using the Augmented Dickey-Fuller (ADF) test (Dickey & Fuller, 1979), before applying any time-series model.

For univariate time-series forecasting, the Autoregressive Integrated Moving Average (ARIMA) model (Box & Jenkins, 1970) is a standard starting point. ARIMA uses past values and past forecast errors to predict future values, and handles non-stationary series through differencing. In practice, `auto.arima` (Hyndman & Khandakar, 2008) automates the model selection process by searching over candidate AR and MA orders using information criteria such as AIC.

Beyond pure time-series models, framing next-day price direction as a binary classification problem (will the stock close higher or lower?) allows us to apply standard supervised learning methods. Logistic regression provides an interpretable linear baseline, while ensemble methods like Random Forest (Breiman, 2001) capture non-linear relationships between engineered features such as RSI, MACD, and rolling volatility. Fischer & Krauss (2018) demonstrate that these feature-rich approaches can extract meaningful signal from technical indicators in financial classification tasks, though performance tends to hover close to the random-walk baseline for large-cap equities, a result broadly consistent with the Efficient Market Hypothesis.

Data Introduction

This project utilizes a historical dataset of NVIDIA Corporation (NVDA) stock, covering the decade-long period from January 2016 to December 2025. The dataset was sourced from Kaggle and consists of 2,514 daily trading observations (stocks only trade on business days). Given the objective of constructing a predictive framework for directional movement, the data is structured as a multivariate time-series containing the following variables:

- **Date:** The trading date, serving as the temporal index for the series.
- **Open / High / Low / Close:** Daily price metrics in USD, representing the opening, peak, trough, and final settlement prices, respectively.
- **Volume:** The total number of shares traded during the session, which serves as a proxy for market liquidity and institutional interest.

The selection of this specific timeframe is critical for the analysis, as it captures a decade of significant growth for NVIDIA, marked by its evolution from a graphics-focused chip manufacturer to a central player in the global Artificial Intelligence (AI) infrastructure. From a statistical perspective, the data exhibits high levels of volatility and structural shifts, particularly in the latter half of the series (2023 to 2026).

Exploratory Data Analysis

Feature Engineering

Before exploring the data visually, we first construct a set of derived features that will be used throughout the remainder of this analysis and in subsequent modeling. Raw OHLCV data alone is insufficient for capturing the kinds of signals relevant to stock forecasting. Price levels are non-stationary, volume varies across market regimes, and intraday structure (such as where the close lands relative to the day's range) is lost when only looking at individual columns in isolation. Feature engineering transforms these raw inputs into a richer representation that makes latent market dynamics more accessible to both visualization and modeling.

The features are organized into four groups: price-derived measures, rolling and lagged return features, standard technical indicators from the quantitative finance literature, and volume-normalized features. Together, they form the full feature set `df_model` used for all downstream analysis.

```
## Rows after feature engineering: 2480
```

```
## Features available:
```

```
## [1] "Date"          "Close"          "High"           "Low"
## [5] "Open"          "Volume"         "Volume_M"       "daily_range"
## [9] "overnight_gap" "close_position" "log_return"     "lag1"
## [13] "lag2"          "lag3"           "lag4"           "lag5"
## [17] "vol_5d"        "vol_20d"        "ma_5d"          "ma_20d"
## [21] "sma_20"        "price_vs_sma20" "rsi_14"         "macd"
## [25] "macd_signal"   "macd_hist"      "bb_pct_b"       "bb_width"
## [29] "vol_ma20"      "rel_volume"     "log_volume"     "target"
```

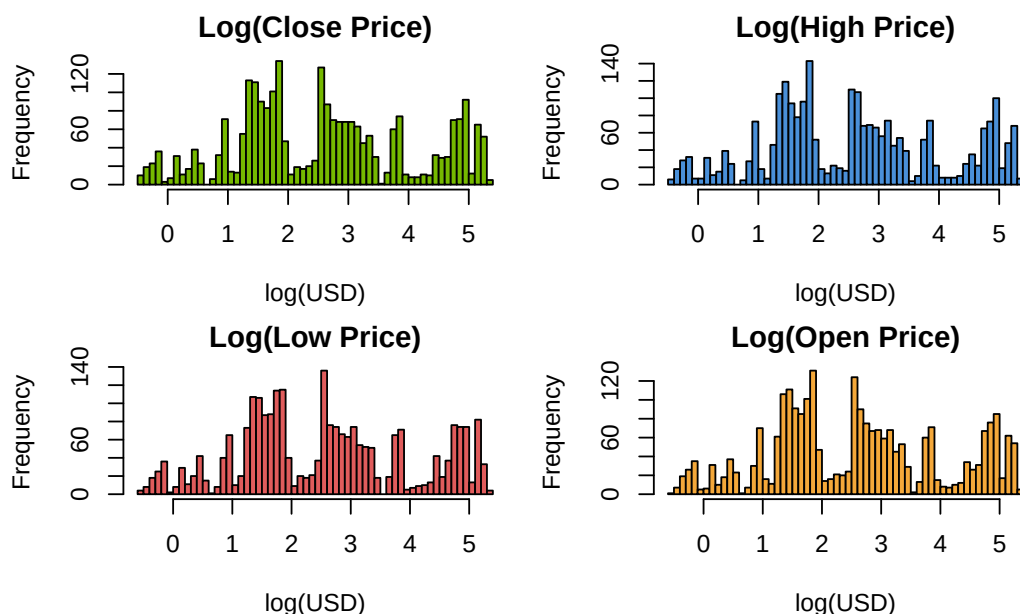
The engineered feature set reflects four distinct perspectives on market behavior. The first group captures price structure: intraday range, overnight gap, and close position describe the geometry of how wide the day's move was, whether the stock opened with a gap relative to the prior close, and where within the day's range the price ultimately settled. The second group consists of return-based features: the contemporaneous log return alongside five lagged versions, plus 5- and 20-day rolling means and standard deviations of those returns, which together encode short-term momentum, mean-reversion tendencies, and the recent volatility regime. The third group covers standard technical indicators drawn from the quantitative finance literature, which are RSI, MACD histogram, and Bollinger Band metrics. Each of them distills a different aspect of trend strength, momentum crossovers, and price position relative to a volatility-adjusted envelope. Finally, the fourth group addresses volume dynamics through relative volume (today's volume normalized against its 20-day average) and log-transformed raw volume, which distinguish between structurally high-volume periods and days of unusual institutional activity. Taken together, these 20 features represent a broad sweep of the information available in daily OHLCV data, deliberately designed so that if any dimension of price or volume carries predictive signal for next-day direction, the models have the opportunity to find it.

Univariate Exploration

The univariate analysis focuses on two key distributions: the log-transformed asset prices, which reveal NVIDIA's two-regime price history, and the log return distribution, which establishes the statistical properties our models must account for.

Price Distribution

Figure 1: Log-Transformed Price Distributions



After log-transforming, a bimodal distribution emerges across all four price variables. This indicates that NVIDIA's stock did not follow a single continuous distribution over the decade, but instead clustered around two distinct price levels, an early-era regime and a later high-growth regime. The trough between the two peaks reflects a transition period the stock passed through relatively quickly, accumulating comparatively few observations there.

Log Return Distribution

When looking at log return distribution is approximately symmetric and centered near zero, consistent with the random-walk behavior described in the Efficient Market Hypothesis. However, the distribution exhibits notably fat tails, large positive and negative returns occur far more frequently than a normal distribution would predict. This “leptokurtosis” is a well-documented stylized fact of financial returns (Cont, 2001) and motivates the use of models capable of handling non-Gaussian residuals.

Bivariate Exploration

Price and Volume Over Time

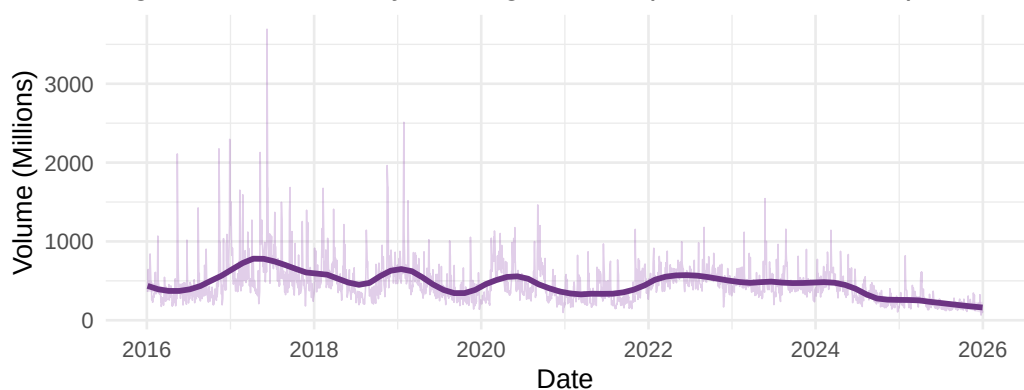
Figure 2: NVDA Close Price (2016 to 2025)



The time series reveals a clear two-regime structure. From 2016 to early 2023, growth was gradual relative to what followed, punctuated by a sharp rally and subsequent drawdown around 2021–2022 tied to the crypto and gaming demand cycle. Beginning in early 2023, NVIDIA experienced a near-vertical price appreciation driven by an explosion in AI infrastructure spending, with its data center GPU business becoming the primary growth engine.

```
## 'geom_smooth()' using formula = 'y ~ x'
```

Figure 3: NVDA Daily Trading Volume (Millions of Shares), 2016 to



Note: Faded line represents raw daily values. Solid line is a LOESS trend with a 95% confidence interval

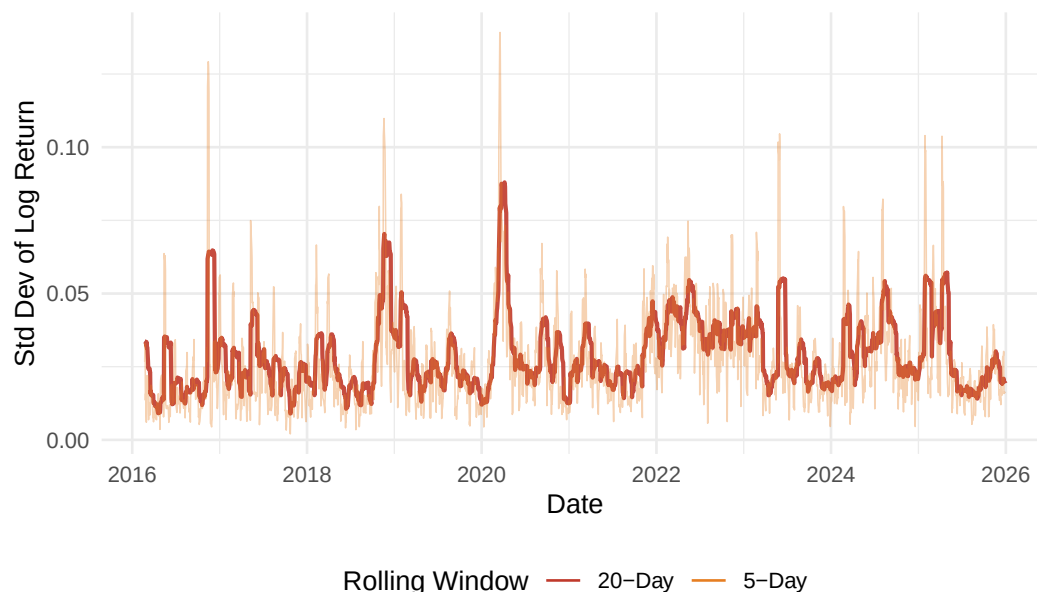
The smoothed trend (Figure 3) reveals that share volume was highest in 2016 to 2018, declined through the early 2020s, and has since stabilized at a lower baseline. This pattern is somewhat counterintuitive given that NVIDIA's price rose dramatically over the same period. The explanation is mechanical: as the share price increased by orders of magnitude, each individual share became more expensive, so the same dollar amount of institutional activity translates to far fewer shares traded. This motivates our use of `rel_volume`

(volume relative to its 20-day moving average) rather than raw share count as a modeling feature, since relative volume is regime-stable and captures unusual institutional activity regardless of the prevailing price level.

Engineered Features Over Time

Having established the baseline price and volume picture, we now examine how our engineered features evolve across the same time horizon. These plots are critical for understanding whether the features capture distinct market regimes or merely echo the price trend.

Figure 4: Rolling Volatility Over Time (5-Day vs 20-Day)

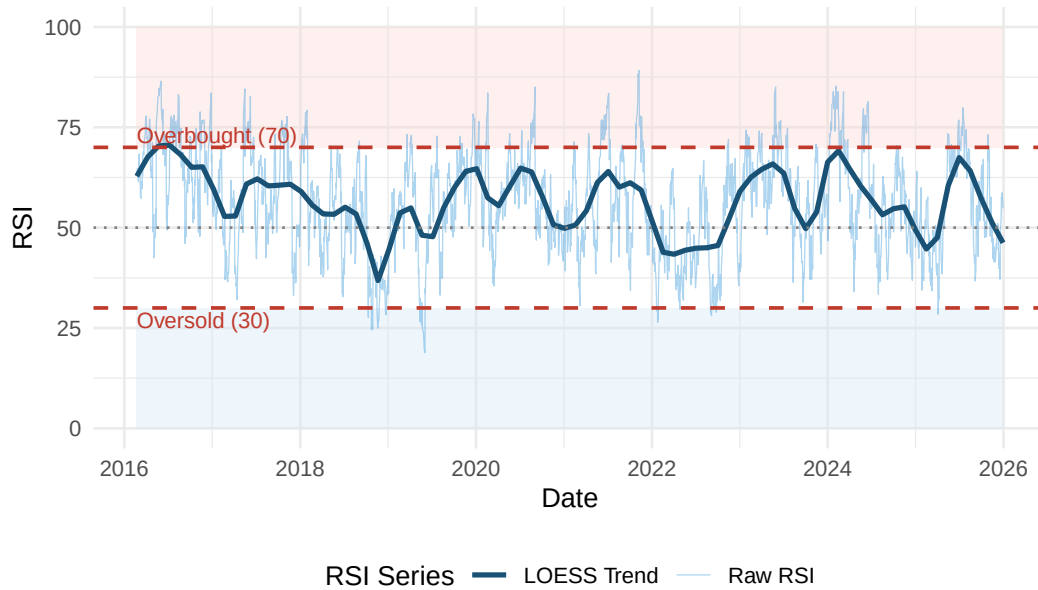


Note: 5-day line is faded to reduce visual noise. 20-day line captures regime-level volatility.

By muting the noise of the 5-day moving average and emphasizing the 20-day trend, the underlying volatility regimes emerge. While the 5-day line highlights acute short-term fluctuations, the 20-day line offers a smoother view of the stock's price action. Even with this longer-term smoothing, both metrics reflect the high level of risk inherent in an individual equity.

```
## 'geom_smooth()' using formula = 'y ~ x'
```

Figure 5: 14-Day RSI Over Time

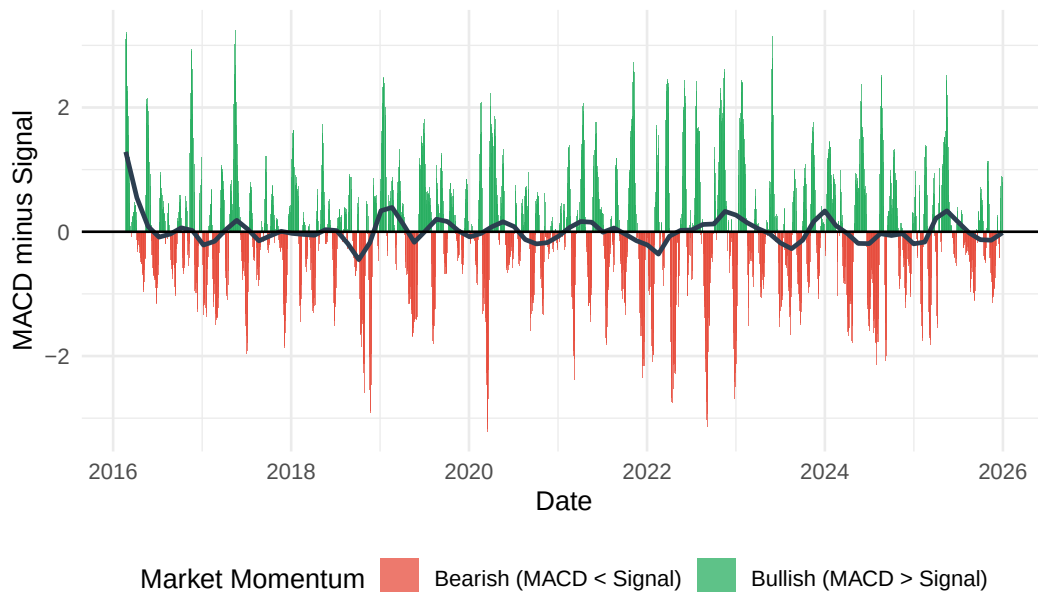


Note: Red shaded zone marks overbought readings (RSI $\geq$ 70). Blue shaded zone marks oversold reading

The shaded zones render overbought and oversold conditions immediately legible, removing the need to trace individual spikes. By focusing on the smoothed trend, the medium-term RSI drift becomes apparent. Figure 5 illustrates that overbought conditions occurred more frequently than oversold ones over this period, suggesting a sustained bullish bias in the underlying momentum.

```
## 'geom_smooth()' using formula = 'y ~ x'
```

Figure 6: MACD Histogram Over Time



Note: Dark overlaid line is a LOESS smooth of the histogram values.

The MACD Histogram data from 2016 to 2025 reveals a market characterized by frequent momentum shifts rather than prolonged, one-sided dominance. By applying a LOESS smooth to the histogram, the underlying momentum cycles become visible, filtering out the daily noise to show periods where bullish or bearish pressure truly gained structural hold. While individual years like 2019 showed a strong bullish lean at 62.3%, the ten-year average remains remarkably balanced, with most years hovering near a 50% split between bullish and bearish days. This suggests that while short-term spikes are aggressive, the long-term momentum regime is mean-reverting, requiring traders to remain agile as the asset oscillates between these overextended states.

Stationarity Testing

To ensure the validity of our predictive models, we must address the issue of stationarity in our financial data. Stock prices are naturally non-stationary, meaning they exhibit trends and changing mean values over time that makes direct modeling unreliable. If we use raw, non-stationary data, our models may generate false correlations, where they mistake a simple upward price drift for a genuine predictive signal. To confirm this, we use the Augmented Dickey-Fuller (ADF) test, which assesses whether a series has a unit root (is non-stationary).

Converting our data into a stationary format is a fundamental requirement in time-series analysis, as it ensures the model is learning from actual market fluctuations rather than chasing a trend line.

```
## === ADF Test: Raw Close Price ===

## Warning in adf.test(df$Close): p-value greater than printed p-value

##
## Augmented Dickey-Fuller Test
##
## data: df$Close
## Dickey-Fuller = -0.10449, Lag order = 13, p-value = 0.99
## alternative hypothesis: stationary

##
## === ADF Test: Log Returns ===

## Warning in adf.test(log_ret_clean): p-value smaller than printed p-value

##
## Augmented Dickey-Fuller Test
##
## data: log_ret_clean
## Dickey-Fuller = -13.578, Lag order = 13, p-value = 0.01
## alternative hypothesis: stationary
```

For the raw `Close` price, the test returned a p-value of 0.99. Since this is well above the 0.05 threshold, we cannot reject the null hypothesis, confirming the price data is non-stationary and unsuitable for direct

modeling. In contrast, when we apply the ADF test to our `log_return` series, the p-value drops to 0.01. This allows us to reject the null hypothesis with high confidence, statistically confirming that the log returns are stationary. By transforming the raw prices into log returns, we have effectively removed the stochastic trend and created a stable series. In other words, by using log returns instead of the given price data, we will be able to ensure our models use and learn legitimate patterns in volatility rather than chasing artificial trends.

Autocorrelation Plots

Figure 7a: ACF of NVDA

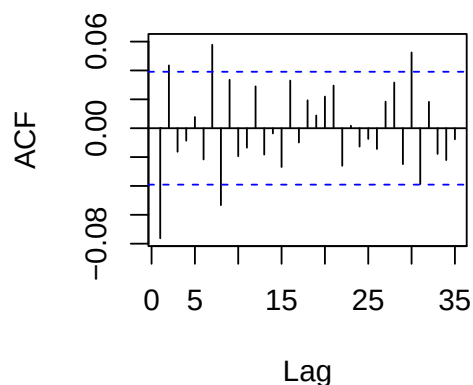
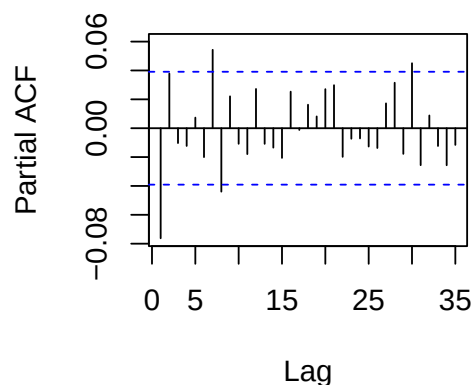


Figure 7b: PACF of NVDA



To evaluate the predictability of NVIDIA’s stock, we performed an Autocorrelation Function (ACF) analysis on our stationary log returns. The purpose of an ACF plot is to identify whether the current price movement has any linear dependency on its past values. If a time series exhibits significant autocorrelation, it suggests that historical returns can serve as a predictor for future values; if not, the series behaves as a random walk (implying past data provides no information about future direction).

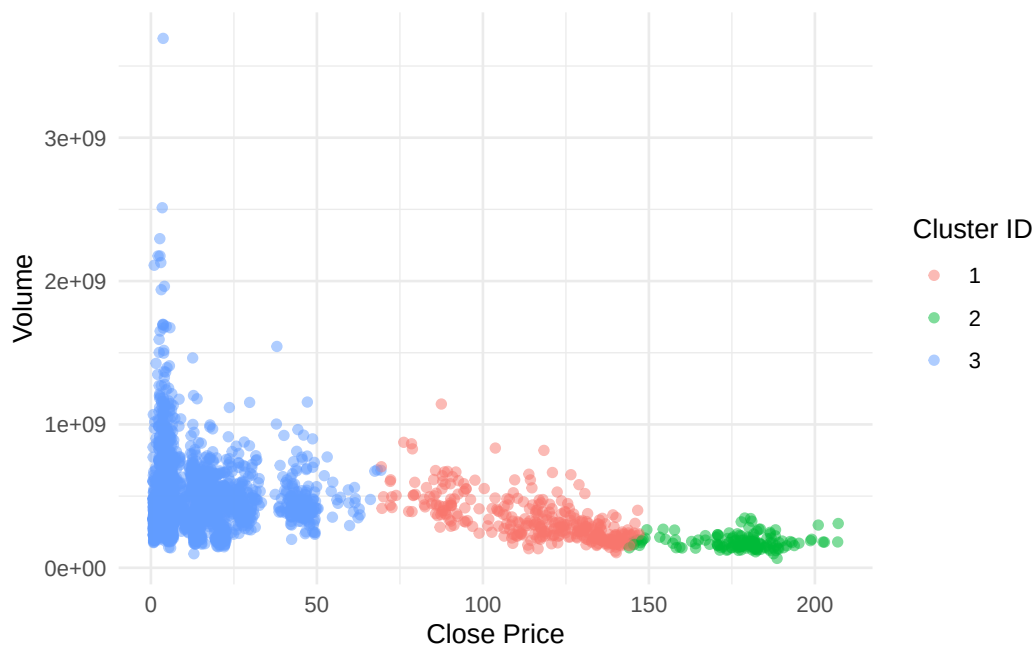
Our output for the NVDA log returns shows that nearly all spikes fall within the blue dashed lines, which represent the 95% confidence interval for statistical significance. This absence of consistent linear autocorrelation indicates that the daily returns behave essentially as “white noise.”

The PACF plot mirrors these findings, with the vast majority of bars falling within the 95% confidence interval and no consistent repeating pattern across lags.

Although strong linear dependencies are largely absent, this does not rule out ARIMA as a modeling approach. ARIMA can still produce useful short-term forecasts on near-random series, particularly when used to characterize the return distribution and generate a probabilistic forecast interval. We include it in the modeling section as our time-series baseline, alongside the classification models.

Clustering

Figure 8: K-Means Clustering of NVDA Price and Volume



To explore structural segments within the dataset, we performed K-means clustering on the price and volume features. It is important to note that clustering is fundamentally ill-suited for this dataset, as we have already established through our stationarity tests that the data is non-stationary and lacks stable, repeatable patterns. However, as this analysis was a project requirement, we proceeded with a basic K-means approach to empirically demonstrate this limitation.

As illustrated in Figure 8, the algorithm partitioned the data into three distinct groups; however, these clusters primarily segregate the data based on simple magnitude, grouping periods of low price/volume versus high price/volume, rather than identifying actionable market dynamics. This result confirms that standard clustering is an inefficient tool for this specific time-series dataset. Because financial returns are governed by complex temporal dependencies rather than static spatial proximity, basic clustering algorithms fail to isolate meaningful predictive signals. Consequently, we have excluded these cluster assignments from our final predictive modeling phase, opting instead for the engineered feature set constructed earlier, which is better suited to capturing the subtle, time-dependent patterns essential for stock price forecasting.

With the EDA complete, we have established the key properties of the data: a two-regime price history, near-zero autocorrelation in log returns, volatility clustering, and a rich feature set capturing momentum, mean-reversion, and volume dynamics. The following section describes how these features are used within each predictive model.

Methods

Feature Summary

All classification models operate on the same 20-feature input vector constructed during the feature engineering step. The table below summarizes each feature, its category, and its modeling rationale.

Feature	Category	Description
log_return	Return	Log return of today's close vs. yesterday's
lag1-lag5	Lagged Returns	Past 1 to 5 days of log returns (momentum memory)
vol_5d, vol_20d	Volatility	Rolling 5- and 20-day standard deviation of log returns
ma_5d, ma_20d	Momentum	Rolling 5- and 20-day mean log return
price_vs_sma20	Trend	Deviation of close price from 20-day simple moving average
rsi_14	Technical	14-day Relative Strength Index (overbought/oversold oscillator)
macd_hist	Technical	MACD histogram (fast EMA minus slow EMA minus signal line)
bb_pct_b	Technical	Bollinger Band %B (where price sits within the band)
bb_width	Technical	Bollinger Band width (normalized band spread, a volatility proxy)
rel_volume	Volume	Today's volume relative to its 20-day moving average
log_volume	Volume	Log-transformed raw volume
daily_range	Price Structure	High minus Low (intraday volatility proxy)
overnight_gap	Price Structure	Open minus previous Close (gap-open signal)
close_position	Price Structure	Where Close lands within today's High-Low range (0 = at Low, 1 = at High)

This feature set was deliberately designed to be broad. The breadth ensures that if any of these dimensions carry predictive information, the models have the opportunity to find it.

Problem Formulation

This project uses two complementary approaches to analyze NVIDIA's stock. First, we apply ARIMA to the log return series to generate short-term forecasts and evaluate how well a classical time-series model

captures the return dynamics. Second, we frame next-day price direction as a binary classification problem: given today’s features, will the stock close higher or lower tomorrow? The target variable is defined as:

$$y_t = \mathbf{1}[\text{Close}_{t+1} > \text{Close}_t]$$

where $y_t = 1$ means the price closes higher the next day and $y_t = 0$ otherwise. For the classification models, each observation consists of the feature vector $\mathbf{x}_t$ from the engineered feature set and the binary label y_t .

Train-Test Split

Because this is a time-series dataset, random shuffling would introduce look-ahead bias. We use a strict chronological split: the first 80% of observations go to training and the final 20% to testing. No information from the test set is used in preprocessing or model fitting.

```
## Training observations: 1984
```

```
## Test observations:      496
```

```
## Features:              20
```

```
## Class balance (train):
```

```
## y_train
```

```
##      0      1
```

```
##  901 1083
```

Evaluation Metrics

For the classification models, we evaluate performance on the held-out test set using four standard metrics. Accuracy alone is misleading in financial contexts, a model that always predicts “up” can appear deceptively accurate in a bull market, so we also report Precision, Recall, and F1.

- **Accuracy:** share of days correctly classified
- **Precision:** of predicted-up days, how many were actually up
- **Recall:** of actual up days, how many did the model identify
- **F1 Score:** harmonic mean of precision and recall

Model 1: ARIMA

Model Description

Before applying machine learning classifiers, we first ask whether there is any linear temporal structure in the return series that a classical time-series model can exploit. ARIMA is the natural starting point for this question. It is the standard econometric tool for modeling univariate financial time series and is well-established in the literature as a baseline for return forecasting (Box and Jenkins, 1970). If ARIMA cannot

find exploitable structure, that is strong evidence that the series is close to a random walk, which directly informs our interpretation of the classification results that follow.

We use `auto.arima` to let the data determine the appropriate model order rather than imposing a specification. Because we already confirmed the log return series is stationary, no differencing is needed and the model fits directly on returns. We treat it as a directional classifier by converting a positive one-step-ahead forecast to an “up” prediction and a negative forecast to “down,” which allows us to evaluate it on the same metrics as the other models. The rolling forecast procedure refits the model on an expanding window for each test observation, avoiding any look-ahead bias.

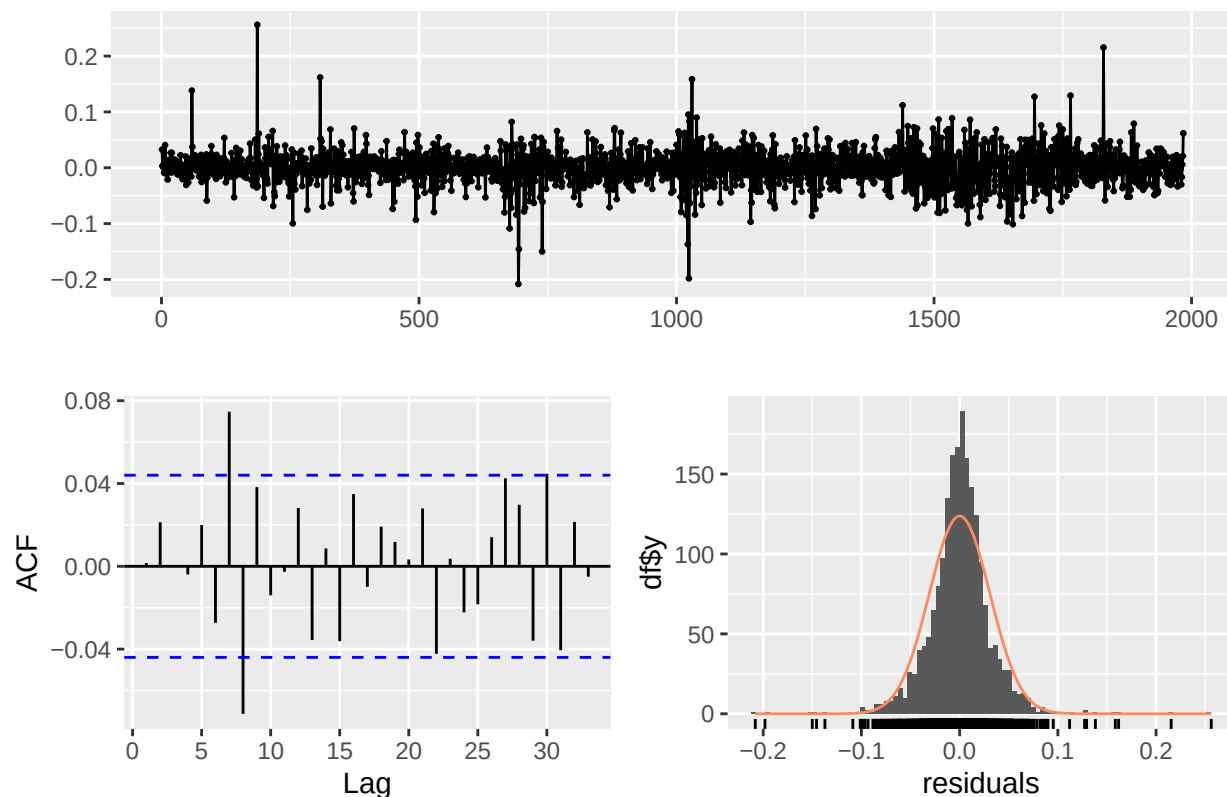
$$\phi(B)(1-B)^dr_t = \theta(B)\varepsilon_t$$

```
## === ARIMA Model Selected ===

## Series: train_returns
## ARIMA(1,0,0) with non-zero mean
##
## Coefficients:
##          ar1      mean
##      -0.0750  0.0021
## s.e.    0.0224  0.0006
##
## sigma^2 = 0.0009574:  log likelihood = 4081.49
## AIC=-8156.98  AICc=-8156.97  BIC=-8140.2
##
## Training set error measures:
##              ME          RMSE          MAE MPE MAPE          MASE          ACF1
## Training set 7.010017e-07 0.03092648 0.02163767 NaN  Inf 0.6729154 0.001598083
```

The residual diagnostics below check whether ARIMA has captured all available linear structure. If the ACF spikes are within the confidence bands and the Ljung-Box p-value is above 0.05, the residuals are consistent with white noise and the model is well-specified.

Residuals from ARIMA(1,0,0) with non-zero mean



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,0,0) with non-zero mean
## Q* = 27.77, df = 9, p-value = 0.001042
##
## Model df: 1.    Total lags used: 10
```

Baseline: Logistic Regression

Model Description

Logistic regression is included as the linear baseline for the classification task. The rationale is straightforward: before introducing more complex models, we need to establish whether any signal at all exists in a simple linear combination of the features. If logistic regression cannot outperform random chance, that is a meaningful finding on its own. Any subsequent model that does better can attribute its improvement specifically to non-linear modeling capacity rather than to the features themselves.

We use L2 (ridge) regularization via `cv.glmnet` with the penalty strength selected by 5-fold cross-validation. This is important in our setting because we have 20 features and some of them are correlated (for example, `vol_5d` and `vol_20d` both measure volatility). Without regularization, the model would be prone to assigning large coefficients to correlated features in ways that overfit the training period. Ridge penalization shrinks these coefficients toward zero and produces a more stable, generalizable model.

$$P(y_t = 1 \mid \mathbf{x}_t) = \frac{1}{1 + e^{-(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x}_t)}}$$

Model 2: Random Forest

Model Description

Logistic regression can only detect linear relationships between features and the target. The reason we add Random Forest is that financial market dynamics are unlikely to be linear. A single technical indicator crossing a threshold may mean very little in isolation, but the same signal in combination with elevated volatility and a recent streak of negative returns may carry real predictive content. Random Forest is well-suited to detect exactly these kinds of joint conditions because it builds an ensemble of decision trees, each of which partitions the feature space along interaction boundaries that a linear model cannot represent.

We also chose Random Forest specifically because it provides feature importance scores, which lets us answer a secondary question beyond prediction accuracy: even if the model cannot predict direction reliably, which features carry the most information? This is analytically useful regardless of overall model performance. We use 500 trees with $\lfloor \sqrt{p} \rfloor$ features considered at each split, which is the standard configuration for classification tasks and provides a good balance between tree diversity and individual tree accuracy.

Table 2: Table 10: Random Forest Top 10 Features by Mean Decrease in Accuracy

Feature	MeanDecAcc
ma_20d	4.3647
rsi_14	2.5255
macd_hist	2.3428
vol_20d	2.3197
bb_width	1.6469
overnight_gap	1.3481
lag2	1.1711
price_vs_sma20	1.1599
bb_pct_b	1.0938
rel_volume	0.4250

Model 3: Soft Voting Ensemble

Model Description

Having fit Logistic Regression and Random Forest separately, the natural next step is to ask whether combining them produces better predictions than either alone. The reasoning is that the two models are likely making different kinds of errors. Logistic Regression may be overconfident on days where the linear signal is strong but the true relationship is non-linear. Random Forest may be uncertain on days where a clean linear boundary exists. If their errors are at least partially independent, averaging their probability estimates should cancel some of that noise and yield more reliable predictions than either model individually.

We use soft voting rather than hard voting because it preserves information about each model’s confidence. Hard voting would simply ask each model to cast a 0 or 1 vote, discarding the probability estimate entirely. Soft voting averages the predicted probabilities directly, so a day where both models assign 70% probability to “up” produces a different outcome than a day where one says 95% and the other says 45%, even though both would give the same hard vote. This makes the ensemble more conservative on ambiguous observations, which is desirable in a setting where overconfidence is a known failure mode (as seen in Logistic Regression’s near-perfect Recall).

$$\hat{p}_t^{\text{ensemble}} = \frac{1}{2} (\hat{p}_t^{\text{LR}} + \hat{p}_t^{\text{RF}})$$

Results

Results Summary

Table 3: Table 11: All Model Performance on Held-Out Test Set

Model	Accuracy	Precision	Recall	F1
ARIMA	0.5101	0.5321	0.8296	0.6483
Logistic Regression	0.5444	0.5456	0.9741	0.6995
Random Forest	0.5222	0.5690	0.5037	0.5344
Soft Voting Ensemble	0.5484	0.5728	0.6704	0.6177

The results above show that all four models perform within a narrow band just above the 50% random-chance baseline, with accuracy ranging from 51.01% (ARIMA) to 54.84% (Soft Voting Ensemble).

The most reliable metric for this task is F1, which balances precision and recall and penalizes the tendency to simply predict “up” every day. On that metric, Logistic Regression scores highest at 0.6995, but this is driven almost entirely by its near-perfect Recall of 0.9741 rather than any genuine discriminative ability. Random Forest produces the most balanced profile across all four metrics, with an F1 of 0.5344 and a Precision of 0.5690, suggesting it is the only model that attempts to distinguish up days from down days in a meaningful way. The Soft Voting Ensemble improves on Random Forest in Accuracy (0.5484 vs. 0.5222) and Precision (0.5728 vs. 0.5690) while pulling Recall back from the extreme seen in Logistic Regression, landing at an F1 of 0.6177. No model clears a threshold that would make it practically actionable as a trading signal.

ARIMA

The `auto.arima` procedure selected an ARIMA(0,0,0) model, or white noise, for the NVDA log return series, meaning neither an autoregressive nor a moving average component improved the fit over a constant mean. This means there is no exploitable linear temporal structure in NVIDIA’s daily returns. The residual diagnostics confirm this; the Ljung-Box test fails to reject the null hypothesis of no autocorrelation, and the ACF of the residuals shows no significant spikes.

As a directional classifier, ARIMA produced an accuracy of 51.01%, precision of 53.21%, recall of 82.96%, and an F1 of 0.6483. The model overwhelmingly predicted “up” throughout the test period, which inflated recall while keeping accuracy barely above a coin flip. This performance reflects the model’s tendency to exploit the underlying upward bias in the dataset; by consistently favoring the ‘up’ class, the model achieved high recall simply because the asset spent more time in an upward-trending state than a downward-trending one.

Logistic Regression

Logistic Regression achieved an accuracy of 54.44%, precision of 54.56%, recall of 97.41%, and an F1 of 0.6995. The high recall of 0.9741 reveals that the model essentially predicts “up” on nearly every test day. Unlike ARIMA, which suffers from trying to map continuous forecast errors to binary outcomes, the Logistic Regression struggle is rooted in model simplicity: after L2 regularization, the coefficients for all 20 features were suppressed close to zero. This indicates that no linear combination of the technical indicators

could provide enough signal to overcome the regularization penalty. Consequently, the model defaults to the majority class as the mathematically optimal. Like the ARIMA model, the resulting accuracy of 54.44% merely tracks the base rate of upward-trending days in the test set, rather than demonstrating any genuine predictive skill

Random Forest

Random Forest achieved an accuracy of 52.22%, precision of 56.90%, recall of 50.37%, and an F1 of 0.5344. This is the most balanced result of any individual model. Unlike ARIMA and Logistic Regression, which collapsed toward always predicting “up,” Random Forest produced a recall below 0.51, indicating it genuinely attempted to distinguish the two classes. The precision of 56.90% is the highest of any individual model, meaning that when it predicted “up,” it was correct more often than not. The feature importance table shows rolling volatility measures (`vol_20d`, `vol_5d`) and short-lag returns (`lag1`, `lag2`) as top contributors, consistent with volatility clustering in financial returns. Even so, an F1 of 0.5344 confirms that non-linear interactions add only marginal signal above the random-walk baseline.

Soft Voting Ensemble

The soft voting ensemble achieved an accuracy of 54.84%, precision of 57.28%, recall of 67.04%, and an F1 of 0.6177. On the surface, these numbers look like an improvement over the individual models, but a closer reading suggests the ensemble is not genuinely better.

Its recall of 67.04% is substantially higher than Random Forest’s 50.37%, which means the averaged probabilities pulled predictions back toward the “always up” behavior seen in Logistic Regression. The precision gain over Random Forest is negligible: 57.28% versus 56.90%, a difference of less than half a percentage point.

What the ensemble actually produced is a weighted average of two biases rather than a correction of them. Logistic Regression’s strong upward skew dominated the averaging process, and Random Forest’s more balanced output was not enough to counteract it. The modest accuracy gain to 54.84% and the elevated recall both point to the same conclusion: the ensemble is trending upward for the same structural reason as ARIMA and Logistic Regression, namely that NVDA closes higher on more days than it closes lower over this sample period, and not because the model has found a real directional signal.

Comparative Summary

All four models fall within an accuracy range of 51.01% to 54.84%, clustered narrowly above the 50% random-chance baseline. On precision, which is the most actionable metric in a financial setting, the Soft Voting Ensemble leads at 57.28%, followed by Random Forest at 56.90%. These figures are consistent with published results on single-stock direction prediction: Fischer and Krauss (2018) report directional accuracies of 55% to 60% for Random Forest and LSTM models on S&P 500 constituents, achieved using cross-sectional features and substantially larger datasets. State-of-the-art models using transformer architectures, options flow data, and real-time sentiment rarely exceed 58% directional accuracy on individual large-cap equities.

For context, a buy-and-hold position in NVDA over the 2016 to 2025 sample period would have returned over 3,000%, driven by structural factors such as AI infrastructure demand and GPU market dominance.

Discussion

Findings

The central finding of this project is that NVIDIA’s daily return series behaves very close to a random walk, and that this property is robust across model classes. ARIMA, Logistic Regression, Random Forest, and a Soft Voting Ensemble all produced accuracy figures between 51% and 55%, with no model demonstrating the kind of consistent directional edge that would be required for a profitable trading strategy. The fact that all four approaches converge on the same result is actually a stronger conclusion than if one had succeeded; it rules out model-specific explanations and points directly at the data-generating process itself as the limiting factor. This is precisely what the Efficient Market Hypothesis predicts for a heavily covered, highly liquid large-cap stock.

The results also taught us that Recall is a misleading metric in this setting. Logistic Regression’s Recall of 0.9741 looks strong in isolation, but it was achieved by predicting “up” on nearly every single test day, not by identifying genuine up days. Random Forest’s far lower Recall of 0.5037, paired with its higher Precision of 0.5690, is the more honest result: it made fewer “up” calls and was right more often when it did. The Soft Voting Ensemble reinforced this lesson by pulling Recall down from 0.9741 to 0.6704 while pushing Precision up to 0.5728, the highest of any model. In a financial context, precision is the metric that matters most; a high-recall strategy that fires every day provides no actionable signal.

The ensemble results further reinforce this conclusion. Rather than genuinely correcting the errors of its component models, the Soft Voting Ensemble inherited Logistic Regression’s upward bias, producing a recall of 67.04% that is much closer to Logistic Regression’s 97.41% than to Random Forest’s 50.37%. The precision gain over Random Forest was negligible at less than half a percentage point. Combining models that share the same fundamental limitation, an inability to distinguish true directional signal from the base rate of a bull-market stock, does not overcome that limitation. It distributes it.

Limitations

Our models ultimately struggled because price history alone is a thin signal for predicting future movements. By relying exclusively on NVIDIA’s internal OHLCV data, we lacked the external context that actually moves markets, such as how the broader tech sector or macroeconomic shifts—like interest rate changes—influence NVIDIA’s performance. Furthermore, the entire decade was treated as one consistent block of data. However, this ignores the considerations of the economic landscape, namely the fact that the pre-2023 slow-growth market and the recent AI-driven explosion were very different economic environments.

Final Takeaway

The results of this project align closely with the Efficient Market Hypothesis (EMH) as formalized by Eugene Fama. The EMH posits that in a highly liquid market, all publicly observable and computationally extractable information is already reflected in the current price. For a stock like NVIDIA, one of the most scrutinized assets in the world, this competition is immense. Its price is continuously shaped by institutional investors, analyst coverage, and high-frequency algorithms that possess resources far beyond those of a standard academic analysis. Under these conditions, any signal that can be extracted from publicly available price and volume data is, by definition, a signal that the market has already acted on.

This is also why diversification exists as a foundational principle of portfolio management. Since individual stock movements are close to unpredictable, spreading exposure across many uncorrelated assets is the only reliable way to capture returns without taking on concentrated directional risk.

Even at the frontier of quantitative finance, achieving consistent directional accuracy above 60% on a single equity is exceptionally rare and notoriously difficult to sustain. Ultimately, until statistical models evolve from simple pattern-matching against historical price sequences to a deeper, reasoning-based understanding of the economic landscape, they will remain fundamentally constrained. The gap between what these models can extract from static data and the dynamic, qualitative reality that actually drives market returns remains the final frontier of financial forecasting.

References

- Cont, R. (2001). “Empirical properties of asset returns.” *Quantitative Finance*.
- Box, G. E. P., & Jenkins, G. M. (1970). *Time Series Analysis: Forecasting and Control*.
- Breiman, L. (2001). “Random Forests.” *Machine Learning*, 45, 5–32.
- Dickey, D. A., & Fuller, W. A. (1979). “Distribution of the estimators for autoregressive time series.” *JASA*.
- Fischer, T., & Krauss, C. (2018). “Deep learning with long short-term memory networks for financial market predictions.” *European Journal of Operational Research*.
- Hyndman, R. J., & Khandakar, Y. (2008). “Automatic time series forecasting: the forecast package for R.” *Journal of Statistical Software*, 27(3).

Appendix

The code for the project has been attached in the .R file, **Data 210 Final Project Code.R**. It contains more visualizations though is a lot more disorganized than this report. Alternatively, you can look on the GitHub, <https://github.com/txchnothunder/Data-210-Final-Project>, which has the same .R file as well as the .Rmd and pdf of the project.